

Reviewer Pack — Association Scheme Eigenvalue Inverse Proportional to ACC^0 ...

Entry 680124d1df5f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-10 09:56:37 UTC

Association Scheme Eigenvalue Inverse Proportional to ACC^0 Circuit Size for CNF Formulas

Entry ID: 680124d1df5f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-10 09:56:37 UTC

1. Conjecture

Field A (mathematical branch): Association Schemes **Field B** (complexity object): ACC^0 Circuit Size for CNF Formulas

Statement:

For any CNF formula with n variables, the size of the minimal ACC^0 circuit computing it is inversely proportional to the absolute value of the smallest eigenvalue of the association scheme constructed from the formula's clause structure.

Rationale (proposer's reasoning):

Association schemes capture combinatorial regularities that might reflect the structure of CNF formulas, and their eigenvalues could encode information about the complexity of computing the function. By linking eigenvalues to circuit size, we might uncover new structural insights.

Taxonomy category: ACC_LB_via_WILLIAMS (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c3db8528810026ec

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.00	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.0s

5.1 Generated Python source

```
import random
from itertools import combinations, product

def generate_cnf(n, m):
    variables = list(range(1, n + 1))
    clauses = []
    for _ in range(m):
        clause = random.sample(variables, random.randint(1, n))
        clauses.append(clause)
    return clauses

def association_scheme(cnf):
    n = max(max(clause) for clause in cnf)
    adjacency_matrix = [[0] * (n + 1) for _ in range(n + 1)]
    for clause in cnf:
        for literal in clause:
            if literal > 0:
                adjacency_matrix[literal][literal] += 1
            else:
                neg_literal = -literal
                adjacency_matrix[neg_literal][neg_literal] += 1
                for other_literal in clause:
                    if other_literal != literal and other_literal != -literal:
                        adjacency_matrix[abs(literal)][abs(other_literal)] += 1
    return adjacency_matrix

def laplacian(matrix):
    n = len(matrix)
    degree_vector = [sum(row) for row in matrix]
    laplacian_matrix = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if i == j:
                laplacian_matrix[i][j] = degree_vector[i]
            else:
                laplacian_matrix[i][j] = -matrix[i][j]
    return laplacian_matrix

def eigenvalues(matrix):
```

```

n = len(matrix)
if n == 1:
    return [matrix[0][0]]

def determinant(submatrix):
    if len(submatrix) == 2:
        return submatrix[0][0] * submatrix[1][1] - submatrix[0][1] * submatrix[1][0]
    det = 0
    for j in range(len(submatrix)):
        det += (-1) ** j * submatrix[0][j] * determinant([row[:j] + row[j+1:] for row in subma
    return det

def characteristic_polynomial(matrix):
    n = len(matrix)
    if n == 1:
        return [matrix[0][0]]
    else:
        char_poly = []
        for k in range(n, -1, -1):
            coeff = (-1) ** (n - k) * determinant([[matrix[i][j] for j in range(k, n)] for i in
            char_poly.append(coeff)
        return char_poly

def roots(poly):
    if len(poly) == 1:
        return []
    elif len(poly) == 2:
        a, b = poly
        return [-b / a]
    else:
        a, *coeffs = poly
        for r in range(-10, 11): # Simple root finding
            if abs(sum(c * r**i for i, c in enumerate(coeffs))) < 1e-6:
                roots.append(r)
                break
        return roots

char_poly = characteristic_polynomial(matrix)
return roots(char_poly)

def dpll(cnf, assignment={}):
    unit_clauses = [c for c in cnf if len(c) == 1]
    if unit_clauses:
        literal = unit_clauses[0][0]
        polarity = True
        if literal < 0:
            polarity = False
            literal = -literal
        if literal in assignment and assignment[literal] != polarity:
            return False
        assignment[literal] = polarity
    pure_literals = [l for l, count in Counter(lit for clause in cnf for lit in clause).items() if
    if pure_literals:
        literal = pure_literals[0]
        polarity = True
        if literal < 0:

```

```

        polarity = False
        literal = -literal
    if literal in assignment and assignment[literal] != polarity:
        return False
    assignment[literal] = polarity

if not cnf:
    return True
literals = [l for clause in cnf for l in clause]
literal, polarity = random.choice(literals), True
if literal < 0:
    polarity = False
    literal = -literal
assignment[literal] = polarity
new_cnf = []
for clause in cnf:
    if not any(lit in assignment and assignment[lit] != (lit > 0) for lit in clause):
        new_clause = [l for l in clause if l != literal]
        if new_clause:
            new_cnf.append(new_clause)
return dpll(new_cnf, assignment)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    m = random.randint(2 * n, 4 * n)
    cnf = generate_cnf(n, m)
    assoc_scheme = association_scheme(cnf)
    laplacian_matrix = laplacian(assoc_scheme)
    eigenvals = eigenvalues(laplacian_matrix)
    smallest_eigenval = min(eigenvals, key=abs)
    if smallest_eigenval == 0:
        return {
            "metric_name": "product",
            "metric_value": float('inf'),
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "smallest_eigenvalue_is_zero"
        }
    circuit_size = dpll(cnf)
    if circuit_size is None:
        return {
            "metric_name": "product",
            "metric_value": float('inf'),
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "circuit_size_not_computable"
        }
    product = abs(smallest_eigenval) * circuit_size
    k = 0.5 # Empirical constant for the acceptance criterion
    return {
        "metric_name": "product",
        "metric_value": product,
        "instances_tested": 1,
        "conjecture_holds": product <= n**k,
        "counterexample": ""
    }

```

```

}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19,
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{result['metric_name']}\", \"metric_
        results.append(result)

    mean = sum(r['metric_value'] for r in results) / len(results)
    std_dev = (sum((r['metric_value'] - mean)**2 for r in results) / len(results))**0.5
    support_fraction = sum(1 for r in results if r['conjecture_holds']) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    elif any(not r['conjecture_holds'] for r in results):
        counterexample = next(r['counterexample'] for r in results if not r['conjecture_holds'])
        first_failing_seed = next(r['seed'] for r in results if not r['conjecture_holds'])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_fa
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test timed out before producing results, preventing evaluation of support fraction or counterexamples. | next: Run test with extended timeout and targeted CNF instances exhibiting potential eigenvalue-circuit size correlations

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	91602
2	preregistration	ollama_remote	qwen3:8b	0	0	24068

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
3	novelty	ollama_remote	qwen3:8b	0	0	21060
4	novelty	ollama_remote	qwen3:8b	0	0	17878
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14730
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19815
7	judge	ollama_remote	qwen3:8b	0	0	14492

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 203644 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in *research/audit/680124d1df5f.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/680124d1df5f.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/680124d1df5f.tar.gz* (if generated)